Lab 3: Advanced Pandas

This lab will give you a chance to practice and experiment with topics from our second week of Pandas:

- Adding, removing, and modifying columns
- Aggregating data with `.groupby`
- Aggregating data with Pivot Tables
- Joining tables

Preliminaries

As a few quick reminders:

- Make sure your code exactly conforms to specifications (e.g. exactly matching variable names)
- Make sure your code works with any provided test code, and do NOT modify that test code unless explicitly stated
- When answering a specific question, ensure that all your data is within the relevant block. For text-based answers, you will be asked to edit a specific markdown cell. For code-based answers, please make sure all code is wholly contained within the relevant `BEGIN` and `END` comments.
- Please be sure to run the first cell (to import libraries) any time you work on your code. Please be sure to run all cells, from start to finish, before you submit. Make sure they run without error, that all provided tests pass, etc.

Information:

- This lab is due Friday 2/13 at 5pm. You will likely have time in class on Monday and/or Wednesday and will likely be able to work on questions 1 and 2 after Monday's lecture
- Each person should submit their own lab solo, but you are welcome to check work and bounce ideas with people near you as you go. They should not, however, give you answers and you should not divide and conquer.
- It's OK to ask AI about syntax (e.g. 'explain to me how `ones` works in NumPy), but not to answer questions for you. Since this lab is meant to teach you the syntax, try to learn it from the class notes and demos / links in the class notes, rather than just blindly googling.

A note on imports

If you try to import a package and see a `ModuleNotFoundError`, you may need to install the package. You usually only have to do this once, and then will have access to that

package in future notebooks. To install a package, add a code cell like the following, where you fill in PACKAGENAME, and run it once.

```
%pip install PACKAGENAME
```

In [46]:
```
import matplotlib.pyplot as plt
import pandas as pd
import random
import numpy as np
```

Question 1: Working with a DataFrame

In this question, we'll start by loading a large dataset about Spotify songs. This data set comes from kaggle and was posted in 2022, and you may have already seen it in Project 1. Note that this was scraped from spotify by another person, so we might be skeptical of using it naively for any real analysis -- or we might want to scrape and obtain our own version! But for introducing pandas, the data set gives a lot to play with. The actual dataset has more columns, but we will only load the following (as indicated by the `usecols` argument).

- `artists`: the artists who performed the track. If there are multiple artists, they're separated by a `;`
- `track_name`: the name of the track
- `popularity`: the popularity of the track as of 2002, a number between 0 (least popular) and 100 (most popular). The Kaggle data set describes "The popularity is calculated by algorithm and is based, in the most part, on the total number of plays the track has had and how recent those plays are. Generally speaking, songs that are being played a lot now will have a higher popularity than songs that were played a lot in the past. Duplicate tracks (e.g. the same track from a single and an album) are rated independently"
- `duration_ms`: The length of the track in milliseconds
- `danceability` and `energy`: Scores between 0 and 1 for how danceable and energetic (respectively) each song is; 1 is the highest.
- `tempo`: The tempo in beats per minute
- `track_genre`: the genre

Run the code below to preview the data set.

In [47]:
```
# Indices do not have to be unique
music = pd.read_csv("spotify.csv", usecols=["artists", "track_name",
                                            "popularity", "duration_ms",
                                            "danceability", "energy", "tempo", "tra

music.sample(10)
```

Out[47]:

	artists	track_name	popularity	duration_ms	danceability	energy	t
954	Chord Overstreet	Hold On	35	198853	0.609	0.443	1
71339	AFI	Halloween	0	238106	0.286	0.929	
6645	Fleshgod Apocalypse	In Aeternum - Orchestral Version	19	327013	0.176	0.521	
39325	Hans Zimmer	Chant	20	153933	0.489	0.600	
90742	Enrique Guzman	El mar - Remastered	30	179004	0.462	0.374	1
104998	Ana Torroja;Sasha, Benny y Erik	50 Palabras 60 Palabras o 100 - En Vivo	27	237813	0.413	0.688	1
44213	Audioslave	Show Me How to Live	41	277840	0.433	0.854	
34044	Salim–Sulaiman;Sukhwinder Singh;Marianne D'Cru...	Chak De India	55	283951	0.755	0.894	1
21224	Sean Paul;David Guetta;Becky G	Mad Love	56	199944	0.793	0.824	
100960	NOFX	The Malachi Crunch	32	171400	0.337	0.882	1

1a: First, use the `.describe` method to get summary information about the data set. Then use that information, Boolean masks, and pandas to answer the following questions:

- How many songs are in the data set?
- What is the median danceability?
- What is the most popular solo-authored Taylor Swift song in the data set? Give the song name as a string, using the exact same capitalization/spacing as in the data set.
- How many distinct songs have popularity equal to 100?
- How many unique genres are there in the data set? Hint: use some pandas to get the **unique** values (as in our code demos from last week, with the bold word being a not-subtle hint!) then use `len()`. Don't count them all!!
- How many pop songs in the data set have a tempo strictly greater than 180? Again, you can use some pandas code to avoid counting these by hand.
- Find the most popular song in the data set whose track name has exactly 9 characters in it. Hint: look back to what we did with baby names.

Do your pandas coding in the following cell, then hardcode your answers in the next cell. For instance, if you used some pandas code to get the median danceability and found it to be 0.5 (it's not), you would fill in `median_danceability=0.5` NOT `median_danceability = SOME PANDAS CODE`. Do NOT delete or change variable names.

In [48]:
```
# Use this cell for pandas coding for question 1a.
music.describe()

# number of songs
print("Number of songs: ",len(music))

# median of danceability
print("Median of danceability: ", music['danceability'].median())

# Taylor Swift
print("Most popular TS song: ", music[music['artists'] == 'Taylor Swift']
      .sort_values('popularity', ascending=False)
      .iloc[0]['track_name'])

# Distinct songs that have popularity equal to 100
print("Distinct songs with popularity 100: ",
      music[music['popularity'] == 100]['track_name'].nunique())

# Unique genres
print("Unique genres: ", music['track_genre'].nunique())

# Pop songs with tempo > 180
print("Pop songs above 180 tempo: ",
      len(music[(music['track_genre'] == 'pop') & (music['tempo'] > 180)]))

# Most popular song with 9 character track name
print("Most popular 9-char song: ", music[music['track_name'].str.len() == 9]
      .sort_values('popularity', ascending=False)
      .iloc[0]['track_name'])
```

```
Number of songs:  114000
Median of danceability:  0.58
Most popular TS song:  Don’t Blame Me
Distinct songs with popularity 100:  1
Unique genres:  114
Pop songs above 180 tempo:  33
Most popular 9-char song:  As It Was
```

In [49]:
```
# Use this cell to hardcode your answers
# BEGIN QUESTION 1a
num_songs = 114000
median_danceability = 0.58
most_pop_taylor_swift_song = "Don't Blame Me"
num_pop_100 = 1
unique_genres = 144
pop_songs_above_180 = 33
most_pop_length_9_song = "As It Was"
# END QUESTION 1a
```

Suppose we think of songs that are both highly-danceable and popular as especially fun. Let's find the "top 10 most fun" such songs in a few different ways.

1b: First, let's consider the sum of popularity and danceability. Create a new column `fun1` that is the sum of a songs popularity and danceability. Then use `sort_by` (and one more pandas command) to show the top 10 songs with the highest `fun1` score. Here and throughout this question, it's OK if some songs occur multiple times (i.e. show the top 10 songs, even if they're not all distinct). You might notice something off about the popularity and danceability of the results.

In [50]:
```
# BEGIN QUESTION 1b
music['fun1'] = music['popularity'] + music['danceability']
music.sort_values('fun1', ascending=False).head(10)

# END QUESTION 1b
```

Out[50]:

	artists	track_name	popularity	duration_ms	danceability	energy	temp
20001	Sam Smith;Kim Petras	Unholy (feat. Kim Petras)	100	156943	0.714	0.472	131.12
81051	Sam Smith;Kim Petras	Unholy (feat. Kim Petras)	100	156943	0.714	0.472	131.12
51664	Bizarrap;Quevedo	Quevedo: Bzrp Music Sessions, Vol. 52	99	198937	0.621	0.782	128.03
88410	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.98
67356	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.98
89411	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.98
68303	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.98
20008	David Guetta;Bebe Rexha	I'm Good (Blue)	98	175238	0.561	0.965	128.04
30003	David Guetta;Bebe Rexha	I'm Good (Blue)	98	175238	0.561	0.965	128.04
81210	David Guetta;Bebe Rexha	I'm Good (Blue)	98	175238	0.561	0.965	128.04

1c: Because of the different scales for popularity scores (integers 0-100) and danceability scores (0-1), the above functionally sorts by popularity and then by danceability. For

instance, a song with popularity 90 and the wrost possible danceability (0) would be sorted above a song with popularity 89 and the highest possibility danceability (1). For a more informative ranking, we should normalize the columns so that they are on the same scale.

Create a new column `fun2` that stores

$$\frac{popularity}{100} + danceability.$$

For instance, if a song had danceability .5 and popularity 60, its `fun2` value would be $\frac{60}{100} + .5 = 1.1$. Then use `sort_by` (and one more pandas command) to show the top 10 songs with the highest `fun2` score.

In [51]:
```
# BEGIN QUESTION 1c
music['fun2'] = music['popularity']/100 + music['danceability']
music.sort_values('fun2', ascending=False).head(10)

# END QUESTION 1c
```

Out[51]:

	artists	track_name	popularity	duration_ms	danceability	energy	tempo
68305	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
89407	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
67358	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
88405	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
20410	Nicki Minaj	Super Freaky Girl	92	170977	0.950	0.891	133.010
51458	Nicki Minaj	Super Freaky Girl	92	170977	0.950	0.891	133.010
51216	Eminem	The Real Slim Shady	87	284200	0.949	0.661	104.504
67356	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980
88410	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980
68303	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980

1d: Third, let's suppose we really want songs that are both especially highly popular and highly danceable. E.g. we'd prefer a song with popularity 90 and danceability 0.9 (where both

are high) to one with popularity 100 and danceability 0.85 (where we've compromised danceability for the sake of popularity). To do so, create a new column `fun3` that is the **minimum** of a songs dancability and (popularity/100). E.g. if a song had danceability .5 and popularity 60, its `fun3` value would be $min\{0.5, \frac{60}{100}\} = 0.5$. Then get the top 10 songs with the highest `fun3` score. You might notice some minor differences.

In [52]:
```
# BEGIN QUESTION 1d
music['fun3'] = music[['danceability', 'popularity']].apply(
    lambda row: min(row['danceability'], row['popularity']/100), axis=1)
music.sort_values('fun3', ascending=False).head(10)

# END QUESTION 1d
```

Out[52]:

	artists	track_name	popularity	duration_ms	danceability	energy	tempo
51458	Nicki Minaj	Super Freaky Girl	92	170977	0.950	0.891	133.010
20410	Nicki Minaj	Super Freaky Girl	92	170977	0.950	0.891	133.010
67358	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
68305	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
89407	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
88405	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
57018	Armani White	BILLIE EILISH.	89	99282	0.900	0.509	100.007
51060	Armani White	BILLIE EILISH.	89	99282	0.900	0.509	100.007
20017	Charlie Puth;Jung Kook;BTS	Left and Right (Feat. Jung Kook of BTS)	92	154486	0.881	0.592	101.058
81024	Charlie Puth;Jung Kook;BTS	Left and Right (Feat. Jung Kook of BTS)	92	154486	0.881	0.592	101.058

1e: Fourth, let's be inspired by regression where we squared residuals. Create a new column `fun4` that is the sum of a songs danceability-squared and its (popularity/100)-squared. E.g. if a song had danceability .5 and popularity 60, its `fun4` value would be $0.5^2 + (60/100)^2 = 0.61$. Then get the top 10 songs with the highest `fun4` score.

In [53]:
```
# BEGIN QUESTION 1e
music['fun4'] = music[['danceability', 'popularity']].apply(
    lambda row: row['danceability']**2 + (row['popularity']/100)**2, axis=1
)
music.sort_values('fun4', ascending=False).head(10)
# END QUESTION 1e
```

Out[53]:

	artists	track_name	popularity	duration_ms	danceability	energy	tempo
68305	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
88405	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
89407	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
67358	Bad Bunny;Chencho Corleone	Me Porto Bonito	97	178567	0.911	0.712	92.005
51458	Nicki Minaj	Super Freaky Girl	92	170977	0.950	0.891	133.010
20410	Nicki Minaj	Super Freaky Girl	92	170977	0.950	0.891	133.010
68303	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980
89411	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980
67356	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980
88410	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.980

1f: Finally, let's make what we just did even more extreme. Create a new column `fun5` that is the sum of a songs danceability and its (popularity/100), after each is raised to the 50th power. E.g. if a song had danceability .5 and popularity 60, its `fun5` value would be $0.5^{50} + (60/100)^{50}$. Then get the top 10 songs with the highest `fun5` score.

In [54]:
```
# BEGIN QUESTION 1f
music['fun5'] = music[['danceability', 'popularity']].apply(
    lambda row: row['danceability']**50 + (row['popularity']/100)**50, axis=1
)
```

```
music.sort_values('fun5', ascending=False).head(10)
# END QUESTION 1f
```

Out[54]:

	artists	track_name	popularity	duration_ms	danceability	energy	tem
20001	Sam Smith;Kim Petras	Unholy (feat. Kim Petras)	100	156943	0.714	0.472	131.1
81051	Sam Smith;Kim Petras	Unholy (feat. Kim Petras)	100	156943	0.714	0.472	131.1
51664	Bizarrap;Quevedo	Quevedo: Bzrp Music Sessions, Vol. 52	99	198937	0.621	0.782	128.0
111635	Quantic	Sol Clap	19	232853	0.985	0.651	115.3
66576	That Girl Lay Lay	Medicaid Baby	10	199639	0.984	0.650	117.9
24916	Delano Smith	Inspiration	5	469240	0.983	0.560	124.9
73374	Oliver Schories	Daily Routines	49	412133	0.982	0.478	122.0
13353	dj funk	Bitches	10	162120	0.981	0.776	128.1
66793	Mixx Master Lee	Featuring Mixx Master Lee, Red Rum & J. Smoov ...	7	341933	0.981	0.675	113.9
67356	Manuel Turizo	La Bachata	98	162637	0.835	0.679	124.9

1g: Look at your work above. Are all of these 5 approaches to finding highly popular and danceable songs distinct, or did any get the same top 10? What do you notice about the approach from 1f specifically. Can you think of another way to describe what it is effectively doing? It might help to think about what happens when you take a small number (like .9) and raise it to the 50th power.

Looking at the five methods for identifying popular and danceable songs:

- fun1 is heavily dominated by popularity due to the scale difference (0-100 vs 0-1).
- fun2 normalizes the scales better, producing a more balanced ranking.
- fun3 prioritizes songs that excel in both dimensions equally.
- fun4 gives slightly more weight to extreme values.

- fun5 produces a dramatically different result. When you raise numbers between 0 and 1 to the 50th power, even values close to 1 become extremely small. This means fun5 is effectively filtering for songs where both danceability AND (popularity/100) are extremely close to 1, acting as a very strict filter that only rewards songs that are nearly perfect on both metrics simultaneously.

Question 2: Practice with `groupby`

2a. The following code loads in information about reviews for 3 Broadway musicals. Note that Time Out and The Guardian review on a scale of 0-5 stars, while Entertainment Weekly reviews as letter grades (e.g. "B+"). I have scaled/imputed these ratings to be between 0 and 10. For The Guardian's review of Come From Away, I averaged their review of the Australian and London production, since I did not see a review of the Broadway production with an unambiguous score.

Use `.groupby` and appropriate aggregators to answer the following questions. For some questions, multiple aggregators may be appropriate.

- Which musical had the highest average (i.e.) mean review?
- Which critic (Time Out, Entertainment Weekly, or The Guardian) was the harshest?
- Which musical had the widest range of reviews? Use a lambda function as your aggregator (Hint: we did this in the code demos by defining a `time_range` function. Think about how to do the same thing with a lambda function. Your syntax will be something like `review_df.groupby("Show")[["Rating"]].agg(lambda x: SOMETHING HERE)`).

Please write code (including code for answers to each question) in the `# BEGIN QUESTION 2a` cell, and then edit the markdown cell below with your answers.

In [55]:
```
import pandas as pd

data = {
    'Show': ['Mean Girls', 'Mean Girls', 'Mean Girls',
             'Come From Away', 'Come From Away', 'Come From Away',
             'Operation Mincemeat', 'Operation Mincemeat', 'Operation Mincemeat'],
    'Source': ['EW', 'The Guardian', 'Time Out', 'EW', 'The Guardian', 'Time Out',
    'Rating': [8.8, 6, 8, 9.5, 7, 8, 9.5, 8, 6]
}

review_df = pd.DataFrame(data)
review_df
```

Out[55]:

	Show	Source	Rating
0	Mean Girls	EW	8.8
1	Mean Girls	The Guardian	6.0
2	Mean Girls	Time Out	8.0
3	Come From Away	EW	9.5
4	Come From Away	The Guardian	7.0
5	Come From Away	Time Out	8.0
6	Operation Mincemeat	EW	9.5
7	Operation Mincemeat	The Guardian	8.0
8	Operation Mincemeat	Time Out	6.0

In [56]:
```
# BEGIN QUESTION 2a

avg_by_show = review_df.groupby("Show")[["Rating"]].agg("mean")
print("Highest average:", avg_by_show["Rating"].idxmax())


avg_by_source = review_df.groupby("Source")[["Rating"]].agg("mean")
print("Harshest critic:", avg_by_source["Rating"].idxmin())


range_by_show = review_df.groupby("Show")[["Rating"]].agg(lambda x: x.max() - x.min
print("Widest range:", range_by_show["Rating"].idxmax())
# END QUESTION 2a
```

```
Highest average: Come From Away
Harshest critic: The Guardian
Widest range: Operation Mincemeat
```

Edit this Markdown cell with your answers to 2a, filling in the bullets below

- Musical with the highest average rating: Come From Away
- Harshest critic: The Guardian
- Musical with the widest range of reviews: Operation Mincemeat

Now lets return to our babynames data frame. Run the cell below to load it back in:

In [57]:
```
import pandas as pd # Convention for importing pandas
import numpy as np
import os.path
import zipfile
import matplotlib.pyplot as plt


# No need to understand this code for now - but it opens pa.txt within a .zip folde
# and loads it as a data frame!
local_filename = "namesbystate.zip"
```

```
zf = zipfile.ZipFile(local_filename, 'r')
pa_name = 'PA.TXT'
field_names = ['State', 'Sex', 'Year', 'Name', 'Count']
with zf.open(pa_name) as fh:
    babynames = pd.read_csv(fh, header=None, names=field_names)


babynames
```

Out[57]:

	State	Sex	Year	Name	Count
0	PA	F	1910	Mary	2913
1	PA	F	1910	Helen	1604
2	PA	F	1910	Anna	1534
3	PA	F	1910	Margaret	1311
4	PA	F	1910	Elizabeth	891
...	...	...	...	...	...
216166	PA	M	2024	Zamari	5
216167	PA	M	2024	Zavian	5
216168	PA	M	2024	Zavion	5
216169	PA	M	2024	Zealand	5
216170	PA	M	2024	Zubayr	5

216171 rows × 5 columns

2b: We'll look at a subset of names this time. Create a new data frame, classdf, that only includes entries corresponding to (1) names in the `classnames` list below and (2) years from 2000 on.

In [58]:
```
classnames = ['Alex', 'Allison', 'Andrew','Caroline', 'Cynthia', 'Dante', 'Devon',
              'Jackson', 'Jeremy', 'Jesse', 'Jordan', 'Justin', 'Kate', 'Martin', '
              'Patrick', 'Sushma', 'Zach']
# BEGIN QUESTION 2b
classdf = babynames[(babynames['Name'].isin(classnames)) & (babynames['Year'] >= 20
# BEGIN QUESTION 2b

try:
    if (len(classdf["Year"].unique())==25) and (len(classdf["Name"].unique())==18):
        print("Public test for 2b passed")
    else:
        print("Public test for 2b FAILED")
except:
    print("Public test for 2b FAILED")
```

```
Public test for 2b passed
```

Lets suppose we want to find the "most popular" name in our class in any year since 2000. Run the following cell, which aggregates by "Name" and uses the "max" aggregator, then sorts by count.

In [59]:
```
classdf[["Name", "Year", "Count"]].groupby("Name").agg("max").sort_values("Count",
```

Out[59]:

Name	Year	Count
Andrew	2024	934
Noah	2024	740
Justin	2024	727
Nathan	2024	700
Jackson	2024	547
Jordan	2024	411
Patrick	2024	373
Allison	2024	296
Alex	2024	258
Caroline	2024	199
Jeremy	2024	186
Jesse	2024	159
Devon	2024	151
Dante	2024	101
Kate	2024	75
Martin	2024	45
Ibrahim	2024	44
Cynthia	2024	38

This seems to suggest that every name peaked in 2024, which doesn't seem right. Run the following cell to zoom in on Andrews:

In [60]:
```
classdf[classdf["Name"]=="Andrew"]
```

Out[60]:

	State	Sex	Year	Name	Count
183620	PA	M	2000	Andrew	934
184690	PA	M	2001	Andrew	883
185750	PA	M	2002	Andrew	895
186825	PA	M	2003	Andrew	877
187957	PA	M	2004	Andrew	833
189137	PA	M	2005	Andrew	825
190347	PA	M	2006	Andrew	750
191614	PA	M	2007	Andrew	733
192880	PA	M	2008	Andrew	655
194189	PA	M	2009	Andrew	590
195496	PA	M	2010	Andrew	571
196817	PA	M	2011	Andrew	515
198150	PA	M	2012	Andrew	468
199484	PA	M	2013	Andrew	466
200806	PA	M	2014	Andrew	440
202186	PA	M	2015	Andrew	415
203518	PA	M	2016	Andrew	409
204915	PA	M	2017	Andrew	293
206316	PA	M	2018	Andrew	286
207718	PA	M	2019	Andrew	268
209115	PA	M	2020	Andrew	240
210484	PA	M	2021	Andrew	255
211934	PA	M	2022	Andrew	207
213366	PA	M	2023	Andrew	195
214801	PA	M	2024	Andrew	184

2c. It appears that there were only 184 Andrew's in 2024, and the 934 observation came from 2000. Edit the markdown cell below to explain what is going on here.

The issue is that the `.groupby("Name").agg("max")` operation is finding the maximum value in each column independently, not the row with the maximum count. The code above returns:

- The maximum `Year` value (2024, the latest year in the dataset)
- The maximum `Count` value (934, the highest count for that name across all years)

We've been able to manually determine that the most common name in our data set was "Andrew" in 2000, with 934 ocurrences. However, if we wanted to figure out what year each of the names in our class "peaked", our table from

```
classdf[["Name", "Year",
"Count"]].groupby("Name").agg("max").sort_values("Count",
ascending=False)
```

is not especially useful: we have the peak count for each name, but not the peak year, and manually investigating every name would be quite tedious. Instead, lets consider another approach:

- Sort the `classdf` data frame by count, in descending order
- Use the `first` aggregator to grab the first entry from each name. This should correspond to the first entry in our sorted data frame, which should have the highest count.

2d. Follow the above steps to create `finaldf`, which should have `Name` as index with just two additional columns (renamed to `Peak Year` and `Peak Count` for you). This data frame should have one entry for each of the 18 names from `classnames` that occured at least 5 times in a given year of a given sex since 2000. For each name, you should see the year where it peaked (since 2020) and the count that year.

In [61]:
```
# BEGIN QUESTION 2d
finaldf = classdf.sort_values('Count', ascending=False).groupby('Name')[['Year', 'C
# END QUESTION 2d
finaldf = finaldf.rename(columns={"Year": "Peak Year", "Count": "Peak Count"})
finaldf


try:
    if finaldf.size == 36 and finaldf.loc["Andrew", "Peak Year"]==2000 and finaldf.
        print("Public test for 2d passed")
    else:
        print("Public test for 2d FAILED")
except:
    print("Public test for 2d FAILED")
```

```
Public test for 2d passed
```

Question 3: Baby Names and Pivot Tables

Let's look at our babynames data set, this time to try to understand birth rates overall. In our code demos, we ran the following cell to group by both year and sex:

In [62]:
```
babynames[["Year", "Sex", "Count"]].groupby(["Year", "Sex"]).agg("sum")
```

Out[62]:

		Count
Year	**Sex**	
1910	**F**	31609
	M	13605
1911	**F**	34514
	M	18071
1912	**F**	45792
...	**...**	...
2022	**M**	53492
2023	**F**	46606
	M	51963
2024	**F**	46170
	M	52098

230 rows × 1 columns

We saw that this had a multiindex structure that was a little complicated, and a pivot table was a better way to view it.

3a: Make a pivot table where the rows (index) correspond to year and the columns to sex at birth. Hint: we made this in class!

In [63]:
```
# BEGIN QUESTION 3a
pivot = babynames.pivot_table(index="Year", columns="Sex", values="Count", aggfunc=
# END QUESTION 3a
pivot
```

Out[63]:

Sex	F	M
Year		
1910	31609	13605
1911	34514	18071
1912	45792	35346
1913	52113	41829
1914	66028	55890
...	...	...
2020	48325	53965
2021	48748	55418
2022	47942	53492
2023	46606	51963
2024	46170	52098

115 rows × 2 columns

3b: Quickly inspecting the counts suggests some wonkiness. There appear to be a lot more females than males from 1910, but more males than females from 2024. Use `.loc` to get the exact M/F ratio for 1910 and for 2024. E.g. if there were 20 M one year and 30 F that same year, the ratio would be 20/30.

In [64]:
```
# BEGIN QUESTION 3b
mfratio1910 = pivot.loc[1910, 'M'] / pivot.loc[1910, 'F']
mfratio2024 = pivot.loc[2024, 'M'] / pivot.loc[2024, 'F']
# END QUESTION 3b

print("1910 ratio ", mfratio1910)
print("2024 ratio ", mfratio2024)


try:
    if 0.3 <= mfratio1910 <= 0.5:
        print("Public test for 3b passed")
    else:
        print("Public test for 3b FAILED")
except:
    print("Public test for 3b FAILED")
```

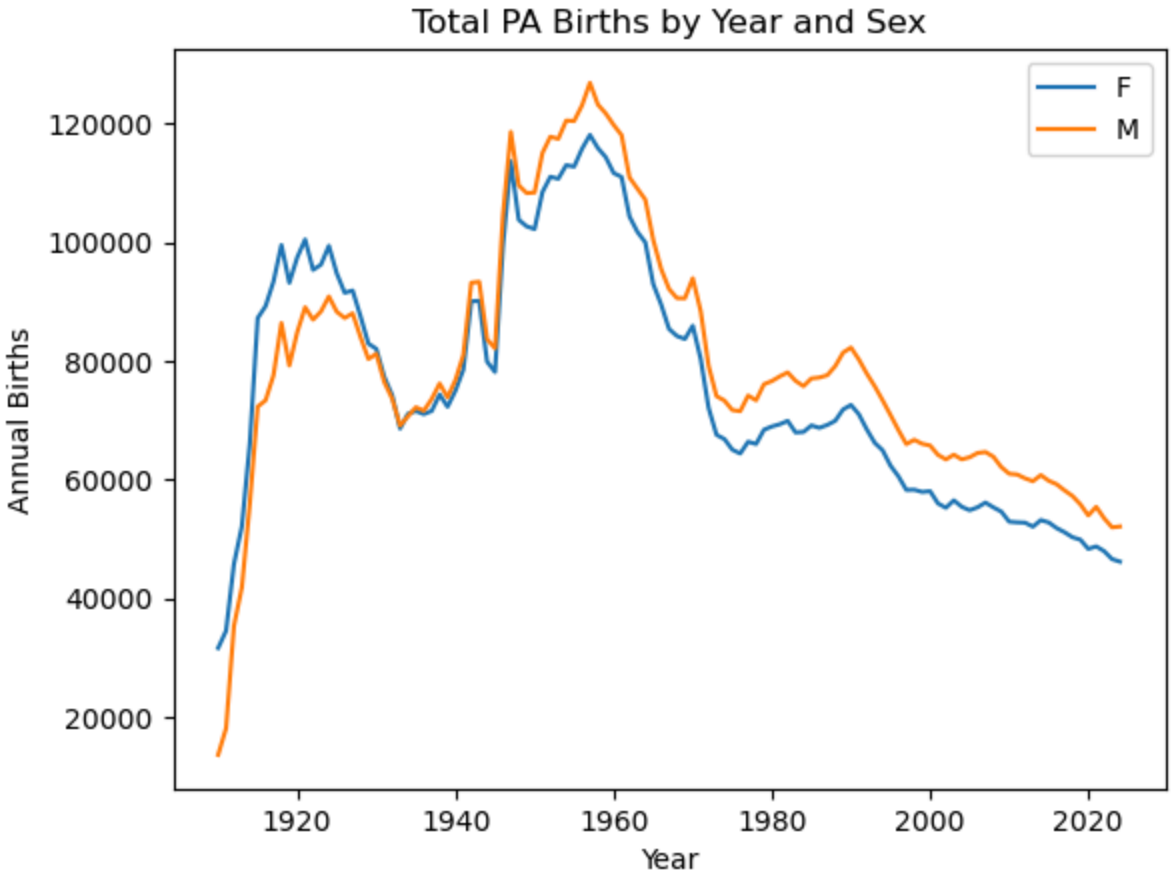
```
1910 ratio  0.4304153880223987
2024 ratio  1.128395061728395
Public test for 3b passed
```

Both of these seem off. Nationally, the ratio of total male births to total female births is about 1.05 (see, e.g., this cdc report).

Instead of just looking at two years, lets visualize the data. Run the following code cell to get a plot

In [65]:
```
pivot.plot()

# Make our plot look more polished
plt.legend()
plt.title("Total PA Births by Year and Sex")
plt.ylabel("Annual Births")
plt.xlabel("Year")
plt.show()
```



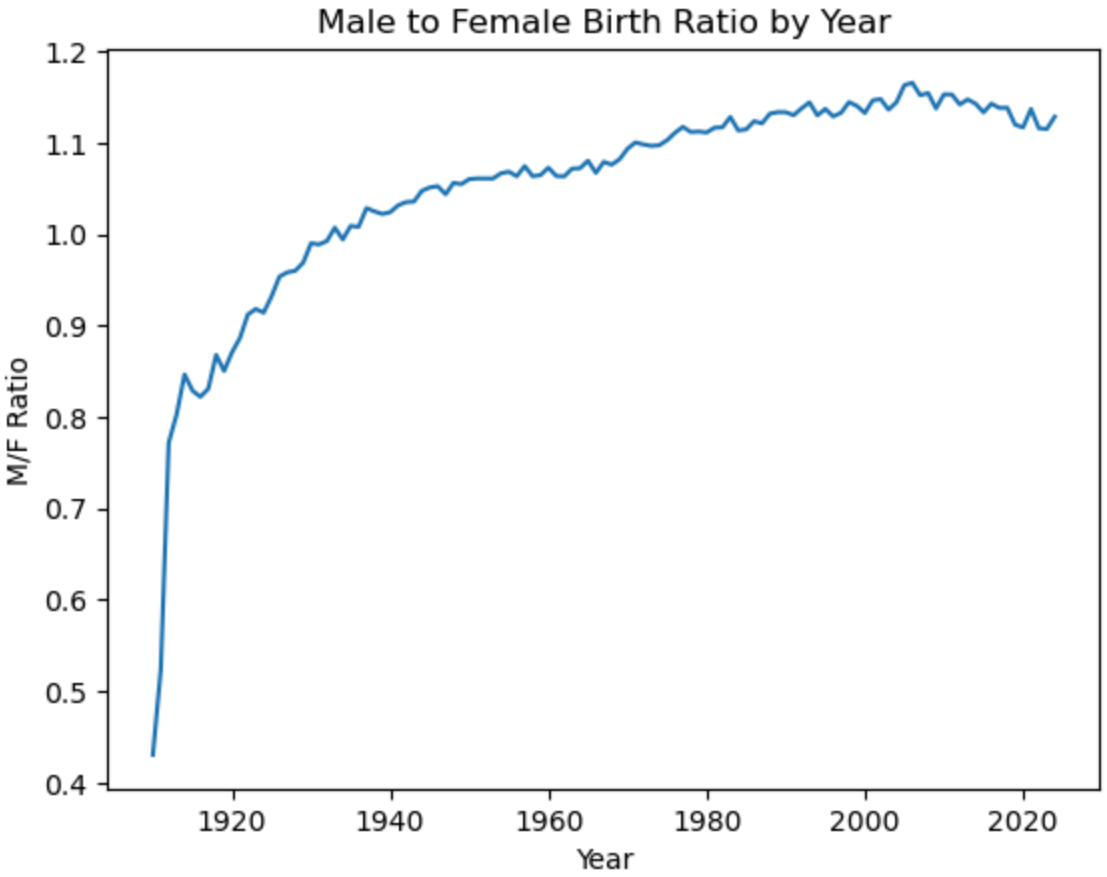
The scale of this data makes it hard to trace whats happening. For instance, we can see that there were more female than male births recorded in our data set early on, but it may not seem as extreme as we observed (i.e. it's hard to see that there were less than half as many male births recorded as female births in 1910).

3c: Create a new column in pivot called `mfratio` that stores the ratios computed earlier, then use matplotlib to plot it. For plotting the data, you should be able to use `plt.plot(pivot["mfratio"])` and then emulate the above to add a title, axis labels, etc

In [66]:
```
# BEGIN QUESTION 3c
pivot["mfratio"] = pivot['M'] / pivot['F']
plt.plot(pivot["mfratio"])
plt.title("Male to Female Birth Ratio by Year")
```

```
plt.ylabel("M/F Ratio")
plt.xlabel("Year")
plt.show()

# END QUESTION 3c
```



The above seems extremely wonky. Let's try to suss out what's going on. First, I asked Google Gemini to explain what was going on and noted that "Early on, F babys outnumber M by a factor of almost 2. But now, M outnumber F (by like 10%)" Here is some of Gemini's response:

> That is a fascinating observation! It’s a perfect example of how data collection practices can create "trends" that aren't actually happening in the real world.
>
> What you are seeing isn't a biological shift in Pennsylvania—it's a change in how the Social Security Administration (SSA) records names.
>
> The "5-Count" Rule
> The SSA only includes a name in the dataset if it appears at least 5 times in a given state for that year. This is done to protect privacy. This rule is the key to your "odd trend."
>
> Why "F" outnumbered "M" (The Diversity Effect)
> Historically, female names have been far more diverse than male

```
names.

The "John" Era: In the early 1900s, a huge percentage of boys were
named one of a few "classic" names (John, William, James). Because
so many boys shared the same few names, almost every boy's name
cleared the "5-count" threshold and was recorded.

The "Unique" Era for Girls: Girls' names were historically more
varied and trend-dependent. Many girls were given names that
occurred only 1, 2, or 3 times.

The Result: A much larger chunk of the "Male" population was
captured in the data, while a significant portion of the "Female"
population was "missing" because their unique names didn't hit the
5-count minimum. This made the Total M count in your data look
higher than Total F.

Why "M" outnumbers "F" now (The Modern Shift)
Over the last few decades, two things happened:

Male Name Diversity exploded: Parents started getting more creative
with boys' names. Now, many more boys' names fail to hit that "5-
count" threshold.

The Biological Baseline: Biologically, about 105 boys are born for
every 100 girls (a ratio of roughly 1.05). As the "diversity gap"
between male and female names closed, the dataset began to reflect
the actual biological reality of births.
```

Some of this is reasonable and may be at play, but some of it is completly inconsistent with the data.

3d: Edit the markdown cell below to explain the major flaw in what Gemini is suggesting. Hint: Gemini is suggesting a potential explanation that would lead to something wildly different. Then do a bit more research and see if you can come up with a better explanation (at least for the portion where Gemini is mistaken).

Gemini's explanation doesn't really make sense: if boys' names were less diverse early on, then more male names would clear the 5-count cutoff, which should make male counts look higher, not lower. The dataset shows females outnumbering males in 1910. A more plausible explanation is that the early SSA name data reflects incomplete and changing record-collection practices, not real birth ratios: reporting and standardization were weaker in the early 1900s, and the dataset's sources likely captured female births more consistently (or male births less consistently) depending on how records were submitted. As the data system became more complete over time, the ratio naturally shifted toward the expected ~1.05 male-to-female birth ratio.

Some of gemini's reasoning related to the distinct number of M/F names. Let's explore this a little more. First, run the code cell below. This should show you that `babynames` has 407 entries from 1910 for babys assigned female at birth. That is, the data set contains 407 distinct names that were used for female babies in 1910:

In [67]:
```
babynames[(babynames["Year"]==1910) & (babynames["Sex"]=="F")]
```

Out[67]:

	State	Sex	Year	Name	Count
0	PA	F	1910	Mary	2913
1	PA	F	1910	Helen	1604
2	PA	F	1910	Anna	1534
3	PA	F	1910	Margaret	1311
4	PA	F	1910	Elizabeth	891
...	...	...	...	...	...
402	PA	F	1910	Theodora	5
403	PA	F	1910	Therese	5
404	PA	F	1910	Verne	5
405	PA	F	1910	Verona	5
406	PA	F	1910	Zelda	5

407 rows × 5 columns

3e: In the cell below do the following:

- Make a pivot table, `pivot2`, that has the rows as indices and the columns as Sex (just like in our previous table `pivot`). However, this time each entry of the pivot table should correspond to the count of distinct names used by the corresponding sex and in the corresponding year. For instance, the top row should be 1910, the first column should be `F`, and the corresponding entry should be 407 like we saw above.
- Merge `pivot` and `pivot2` into one data frame. Hint: Use our syntax from class, specifying `right_on` and `left_on`
- Take a look at maintable. You should, e.g., see the numbers 407 (the total number of distinct female baby names used in 1910 in the data set) and 31609 (the total number of female babys recorded in 1910) in the top row. Briefly explain why the column names have x and y subscripts.

In [68]:
```
# BEGIN QUESTION 3e
pivot2 = babynames.pivot_table(index="Year", columns="Sex", values="Name", aggfunc=
maintable = pd.merge(pivot[['F', 'M']], pivot2, left_index=True, right_index=True)
# # END QUESTION 3e
```

```
try:
    if pivot2.loc[1910, "F"]==407 and maintable.size==460:
        print("Public test for 3e passed")
    else:
        print("Public test for 3e FAILED")
except:
    print("Public test for 3e FAILED")


maintable
```

```
Public test for 3e passed
```

Out[68]:

Sex	F_x	M_x	F_y	M_y
Year				
1910	31609	13605	407	253
1911	34514	18071	431	299
1912	45792	35346	494	401
1913	52113	41829	530	444
1914	66028	55890	594	515
...	...	...	...	...
2020	48325	53965	1637	1373
2021	48748	55418	1624	1442
2022	47942	53492	1623	1431
2023	46606	51963	1641	1428
2024	46170	52098	1598	1430

115 rows × 4 columns

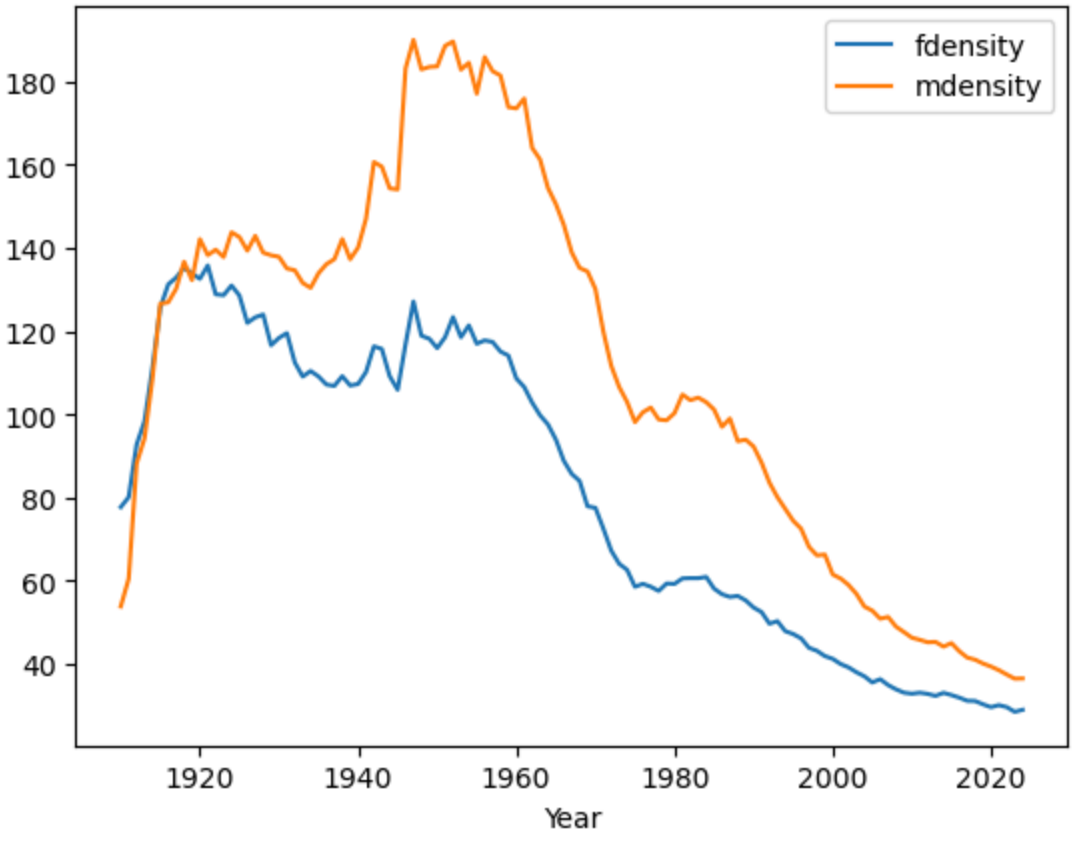
The column names have _x and _y subscripts because both pivot and pivot2 have columns with the same names (F and M). When merging two DataFrames with overlapping column names, pandas adds suffixes to distinguish them.

3f: Lets think about the name diversity mentioned by gemini. Add two more columns to `maintable` : one (called `fdensity`) giving the "name density" of female names each year, and one (called `mdensity`) for male names each year. For instance, in 1910, the female density would be 31609/407=77.66. Then make a plot (in the style of "Total PA Births by Year and Sex") with one line for `fdensity` and one line for `mdensity` . Hint: for this plot, you will probably want to filter down `maintable` to just the two columns you want: remember that `df[LIST OF COLUMNS]` is particularly helpful for selecting just some of the columns! Don't forget to add a title and axis labels to your plot

In [69]:
```
# BEGIN QUESTION 3f
maintable["fdensity"] = maintable["F_x"] / maintable["F_y"]
maintable["mdensity"] = maintable["M_x"] / maintable["M_y"]
# Filter down DF for our plot
maintable[["fdensity", "mdensity"]].plot()
plt.legend()
...
plt.show()
# # END QUESTION 3f


try:
    if np.isclose(maintable.loc[1910, "fdensity"], 77.663391) and maintable.size==6
        print("Public test for 3f passed")
    else:
        print("Public test for 3f FAILED")
except:
    print("Public test for 3f FAILED")

maintable
```




```
Public test for 3f passed
```

Out[69]:

Year	Sex	F_x	M_x	F_y	M_y	fdensity	mdensity
1910		31609	13605	407	253	77.663391	53.774704
1911		34514	18071	431	299	80.078886	60.438127
1912		45792	35346	494	401	92.696356	88.144638
1913		52113	41829	530	444	98.326415	94.209459
1914		66028	55890	594	515	111.158249	108.524272
...		...	...	...	...	...	...
2020		48325	53965	1637	1373	29.520464	39.304443
2021		48748	55418	1624	1442	30.017241	38.431345
2022		47942	53492	1623	1431	29.539125	37.380853
2023		46606	51963	1641	1428	28.400975	36.388655
2024		46170	52098	1598	1430	28.892365	36.432168

115 rows × 6 columns

3g: Finally, we probably could have gotten `fdensity` and `mdensity` a bit easier. Make a new pivot table, `pivot3`, that gets the same information as in the `fdensity` and `mdensity` column directly. As with `pivot` and `pivot2`, this should be a pivot table where the rows correspond to years, and the columns, to sex assigned at birth. You should not do any joining/merging/creating of new columns; create `pivot3` using a single call to `babynames.pivot_table`

In [70]:
```
# BEGIN QUESTION 3g
pivot3 = babynames.pivot_table(index="Year", columns="Sex", values="Count", aggfunc
# # END QUESTION 3g


try:
    if np.isclose(pivot3.loc[1910, "F"], 77.663391) and pivot3.size==230:
        print("Public test for 3g passed")
    else:
        print("Public test for 3g FAILED")
except:
    print("Public test for 3g FAILED")



pivot3
```

```
Public test for 3g passed
```

Out[70]:

Sex	F	M
Year		
1910	77.663391	53.774704
1911	80.078886	60.438127
1912	92.696356	88.144638
1913	98.326415	94.209459
1914	111.158249	108.524272
...	...	...
2020	29.520464	39.304443
2021	30.017241	38.431345
2022	29.539125	37.380853
2023	28.400975	36.388655
2024	28.892365	36.432168

115 rows × 2 columns

Question 4: Practice Merging Data Frames

Suppose we wanted to compute the population density of several states. Run the following two code cells, which load the population (in millions) and area (in square miles) of 5 states. Note that these were generated by gemini, so the data may not be perfect.

In [71]:
```
pop_df = pd.read_csv("pops.csv")
pop_df
```

Out[71]:

	State	Population_Millions
0	CA	39.2
1	CT	3.6
2	NJ	9.3
3	NY	19.8
4	PA	13.0

In [72]:
```
area_df = pd.read_csv("area.csv")
area_df
```

Out[72]:

	State	Land_Area_sq_mi
0	California	155779
1	Connecticut	4842
2	New Jersey	7354
3	New York	47126
4	Pennsylvania	44743

To compute the population density, we want to merge these data frames so that we can divide the population by the land area. Lets try this:

In [73]:
```
combined_df = pd.merge(pop_df, area_df, left_on="State", right_on="State")
combined_df
```

Out[73]:

	State	Population_Millions	Land_Area_sq_mi

4a: In one sentence or phrase, explain why the above process did not work. Edit the following markdown cell with your answer:

The column names in `pop_df` use state abbreviations while the column names in `area_df` use full state names, so there are no matching values to merge on.

The following code cell loads another csv, this time containing a mapping between state names and abbreviations. Run it and take a look.

In [74]:
```
key_df=pd.read_csv("key.csv")
key_df.head()
```

Out[74]:

	State	Abbreviation
0	Alabama	AL
1	Alaska	AK
2	Arizona	AZ
3	Arkansas	AR
4	California	CA

4b: Follow these steps to create a dataframe, `final_df`, that includes just the 5 states (with their full name) and their population density

- Merge `pop_df` and `key_df`, to get a data frame including the state abbreviation, state name, and population of each of our 5 states (it may include additional columns)
- Use the state name column in that new data frame, to merge in `area_df` (so now you should have a data frame including the population and area of each state, including

many other columns)

- Create a new column storing the population density `population/area` (in people per square mile -- not millions of people per square mile)
- Delete superfluous columns and rename any existing columns to create a final data frame, `combined_df`, including just two columns: `State` (giving the full state name) and `PopDensity` (giving the population density)

In [75]:
```
# BEGIN QUESTION 4b
temp_df = pd.merge(pop_df, key_df, left_on="State", right_on="Abbreviation")
temp_df = pd.merge(temp_df, area_df, left_on="State_y", right_on="State")
temp_df["PopDensity"] = (temp_df["Population_Millions"] * 1000000) / temp_df["Land_
combined_df = temp_df[["State", "PopDensity"]].copy()
# # END QUESTION 4b


try:
    if combined_df.size==10 and 200 <= combined_df.loc[0, "PopDensity"] <= 300:
        print("Public test for 4b passed")
    else:
        print("Public test for 4b FAILED")
except:
    print("Public test for 4b FAILED")


combined_df
```

```
Public test for 4b passed
```

Out[75]:

	State	PopDensity
0	California	251.638539
1	Connecticut	743.494424
2	New Jersey	1264.617895
3	New York	420.150236
4	Pennsylvania	290.548242

Finally, there are a lot of different options for merge. The first two cells below load two data frame, but without the exact same information. One loads ID's 1, 2, 3, 4 and gives the name of each. The next loads IDs 1, 2, 3, and 5 and gives the age of each. So, e.g., we know that ID 1 corresponds to Alice whose age 20, but we don't know Dana's age or the name of ID 5.

In [76]:
```
df_names = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'Name': ['Alice', 'Bob', 'Charlie', 'Dana']
})
df_names
```

Out[76]:

	ID	Name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	Dana

In [77]:
```
df_ages = pd.DataFrame({
    'ID': [1, 2, 3, 5],
    'Age': [20, 18, 33, 35]
})
df_ages
```

Out[77]:

	ID	Age
0	1	20
1	2	18
2	3	33
3	5	35

4c: Run the following cells, which explore several different options for how. Then answer the following questions:

- What do each of the how options (inner, left, right, outer, and cross) do?
- Which is the default option for `pd.merge` (hint: think about adding a cell and running `merge` with no `how` argument!)
- How come we did not need to specify a `left_on` or `right_on`?
- Why do all these data frames only have one `ID` column, rather than an `ID_x` and `ID_y` like we saw with `M_x` and `M_y` in one of our above examples?

In [78]:
```
pd.merge(df_names, df_ages, how="inner")
```

Out[78]:

	ID	Name	Age
0	1	Alice	20
1	2	Bob	18
2	3	Charlie	33

In [79]:
```
pd.merge(df_names, df_ages, how="left")
```

Out[79]:

	ID	Name	Age
0	1	Alice	20.0
1	2	Bob	18.0
2	3	Charlie	33.0
3	4	Dana	NaN

In [80]:
```
pd.merge(df_names, df_ages, how="right")
```

Out[80]:

	ID	Name	Age
0	1	Alice	20
1	2	Bob	18
2	3	Charlie	33
3	5	NaN	35

In [81]:
```
pd.merge(df_names, df_ages, how="outer")
```

Out[81]:

	ID	Name	Age
0	1	Alice	20.0
1	2	Bob	18.0
2	3	Charlie	33.0
3	4	Dana	NaN
4	5	NaN	35.0

In [82]:
```
pd.merge(df_names, df_ages, how="cross")
```

Out[82]:

	ID_x	Name	ID_y	Age
0	1	Alice	1	20
1	1	Alice	2	18
2	1	Alice	3	33
3	1	Alice	5	35
4	2	Bob	1	20
5	2	Bob	2	18
6	2	Bob	3	33
7	2	Bob	5	35
8	3	Charlie	1	20
9	3	Charlie	2	18
10	3	Charlie	3	33
11	3	Charlie	5	35
12	4	Dana	1	20
13	4	Dana	2	18
14	4	Dana	3	33
15	4	Dana	5	35

- `inner` : Keeps only rows where the ID exists in both DataFrames (IDs 1, 2, 3). This is the **default** option for `pd.merge` .
- `left` : Keeps all rows from the left DataFrame (`df_names`), filling in NaN for missing values from the right (IDs 1, 2, 3, 4).
- `right` : Keeps all rows from the right DataFrame (`df_ages`), filling in NaN for missing values from the left (IDs 1, 2, 3, 5).
- `outer` : Keeps all rows from both DataFrames, filling in NaN where data is missing (IDs 1, 2, 3, 4, 5).
- `cross` : Creates a Cartesian product of both DataFrames, pairing every row from the left with every row from the right (16 total rows).

We did not need to specify `left_on` or `right_on` because both DataFrames have a column with the same name (`ID`), and pandas automatically uses it as the merge key.

There is only one `ID` column (not `ID_x` and `ID_y`) because `ID` is the **merge key**. Pandas only creates `_x` and `_y` suffixes for non-key columns that have the same name in both DataFrames. Since `ID` is used to join the tables, it appears only once in the result.

Lab Check Hint for Lab 3

For your next lab check, you should feel good about:

- Syntax for creating/modifying a column, like in Question 1
- The results of `.groupby` (with various aggregation functions) on small data frames, like in Question 2a
- The results of `.merge` (with various merge options) on small data frames, like in Question 4

For this lab check, I will not ask you to directly write pandas code, but I may ask you about the output of pandas code.

Submission Instructions

We will submit this lab as a PDF.

1. Hit "Kernel --> Restart & Run All"
2. Make sure that all of your cells run and that any output necessary for grading is shown. Be sure that you pass the provided tests.
3. Create a PDF of your notebook. To do so, (1) hit File -> Download as -> HTML (or potentially Save and File -> Export Notebook As -> HTML, depending on your version of Jupyter/OS) (2) open that HTML file in Google Chrome (I drag from downloads into a new Chrome tab), and (3) save that HTML file as a PDF (I type `ctrl+P` to print and then select `Save as PDF` as my destination). Note that this is a bit more involved than just saving as a PDF directly from Jupyter, but usually leads to better-formatted notebooks so **please follow this process exactly.**
4. Skim the PDF and make sure all your code and written responses are fully visible. If you notice anything (like your code lines are too long and get cut off; your written answers are cut off), fix those issues and repeat the above process. If I cannot see your work, it will not be graded.
5. Upload that PDF to Gradescope. If you run into any issues, please reach out to me before the deadline and document that you've completed everything (e.g. by emailing me a copy of your Jupyter notebook).

In [ ]: